

MANUAL DE PREPARACIÓN · V1.1 · CONSULTA 04 JUN 2026

Guía para MonadBlitz Medellín

Todo lo que el equipo necesita entender —de la blockchain a la línea de comandos— para llegar el sábado con wallets listas, un contrato corriendo en Monad y un pitch que se defienda en 90 segundos.

FECHA

Sáb 06 jun

2026 · 09:00–22:00

SEDE

Indie Universe

Hotel · Medellín

FORMATO

1 día

~7 h de hacking real

PREMIO

USD 2.000

por ciudad

— 01 · EL EVENTO

Qué es MonadBlitz Medellín

Un hackathon de un solo día organizado por Monad Colombia. La premisa publicada es deliberadamente abierta: **«Un día. Sin límites. Solo construye.»**

No es una competencia con un enunciado cerrado tipo «resuelvan este problema». La consigna oficial es: **llega, forma equipo, construye lo que quieras y haz deploy. Si es interesante y corre en Monad, vale.** Esa libertad es una oportunidad y una trampa a la vez: gana quien elige un alcance pequeño y lo termina, no quien sueña en grande y entrega a medias.

I LO QUE DE VERDAD SE EVALÚA

Aunque el reto es abierto, la inferencia fuerte —a partir del formato Blitz global de Monad y de la preparación local previa— es que el jurado quiere ver un **MVP funcional que use Monad de verdad**, no una idea en diapositivas. En la práctica esperan: repo, demo desplegada o reproducible, **dirección del contrato**, una **transacción real visible en el explorer** y un pitch corto que explique el valor para el usuario.

El contexto local que ya existe

Medellín no parte de cero. Hubo preparación previa de la comunidad (DevLabX3 / TdeA) con un **workshop universitario** que dejó sitio oficial, brandkit y un repositorio técnico de referencia. Ese repo usa exactamente el stack que conviene replicar: Solidity, Remix, WalletConnect/RainbowKit, Wagmi, Tailwind y Ethers, con un contrato `TodoList.sol` y un frontend de interacción. **Es nuestra plantilla base.**

— 02 · EL OBJETIVO

Qué significa «estar listos»

Una definición operativa, para no engañarnos. Estamos preparados cuando el equipo puede, sin improvisar:

- ◆ Llegar con **tres wallets de prueba** ya creadas, con la red Monad Testnet agregada y con `MON` del faucet.
- ◆ Correr un **frontend + contrato simple** sobre Monad Testnet, de principio a fin.
- ◆ Explicar en **90 segundos** por qué el proyecto necesita blockchain (no «porque sí»).
- ◆ Dividir el trabajo **sin que los dos compañeros dependan de saber Solidity**.

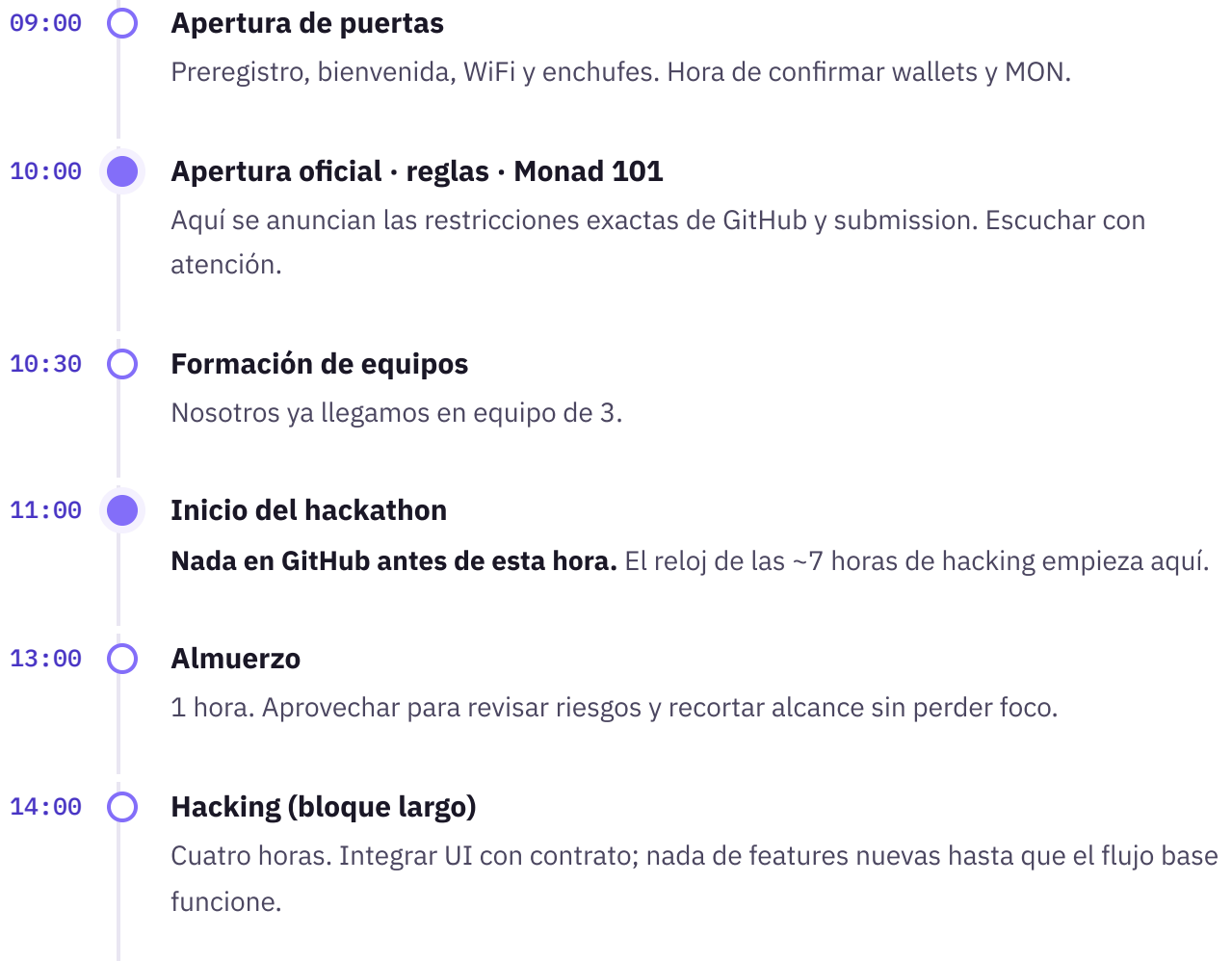
**LA PRUEBA DE FUEGO**

Si un compañero que no sabe blockchain puede abrir la dApp, conectar su wallet y crear un registro que luego aparece en el explorer —sin que nadie le explique nada— estamos listos. Ese es el listón.

03 · EL EVENTO

Agenda y las reglas de GitHub

El día tiene dos fronteras inamovibles: el **inicio a las 11:00** y el **code freeze a las 18:00**. Todo lo demás se mueve alrededor de esas dos líneas.



- 18:00** ○ **Code freeze**
No más cambios en GitHub. Cero commits después de esta hora.
- 18:30** ○ **Deadline de submissions**
Entrega en la app de Monad: nombre, descripción, repo, contrato, demo, integrantes.
- 19:00** ○ **Demos y showcase**
Dos horas. Abrir la demo ya cargada antes de pasar al frente.
- 22:00** ● **Premiación y cierre**
Tras el break y la deliberación del jurado (21:00–22:00).

! LAS DOS REGLAS QUE DESCALIFICAN

1. No debe haber **nada en el repo antes de las 11:00**. Pueden crear la cuenta y el repositorio vacío, pero no subir código del proyecto antes del inicio. **2.** Tras el **code freeze de las 18:00** no se aceptan más cambios. Confirмен la redacción exacta de ambas reglas en la apertura de las 10:00, porque pueden ajustarse.

— 04 · EL EVENTO

Premios y criterios de evaluación

Hay **USD 2.000** en juego por ciudad. Decide un **jurado técnico**; el voto de los participantes en la app de Monad cuenta como señal indicativa, no decisiva.

1º **USD 1.000**
PRIMER LUGAR

2°

USD 700

SEGUNDO LUGAR

3°

USD 300

TERCER LUGAR

Criterios probables del jurado

No están publicados al detalle, pero el patrón Blitz y la naturaleza «funciona en vivo» del evento los hacen predecibles:

CRITERIO	QUÉ BUSCAN EXACTAMENTE
Funciona en vivo	Contrato desplegado, la wallet conecta, la transacción confirma. Sin esto, lo demás no importa.
Usa Monad de verdad	No basta una landing page. Tiene que haber estado on-chain y una tx real en el explorer.
Idea clara	El problema se entiende en 20 segundos. Si hay que explicarlo dos veces, es demasiado complejo.
Demo visual	Aunque decide el jurado técnico, el voto de participantes influye: una demo bonita y entendible suma.
Alcance pequeño pero completo	Mejor un flujo redondo que un protocolo grande a medias.
Seguridad básica	Sin datos personales on-chain, sin private keys en el repo, sin poderes de admin absurdos.
Presentación	Historia + demo + impacto + siguiente paso. En ese orden.

— 05 · FUNDAMENTOS

Blockchain desde cero

En una frase: una **base de datos compartida** donde los cambios importantes se registran como transacciones firmadas, verificables y muy difíciles de alterar. Nadie es dueño del libro; todos tienen una copia idéntica.

La analogía útil es un **libro de cuentas público** que miles de computadoras (los nodos) guardan a la vez. Cuando quieres cambiar algo —enviar dinero, registrar un dato— escribes una línea nueva, la firmas con tu clave privada, y la red verifica que tu firma es válida antes de aceptarla. Una vez escrita y confirmada, esa línea es prácticamente imposible de borrar o falsificar, porque tendrías que convencer a toda la red a la vez.

I POR QUÉ IMPORTA PARA EL PITCH

Blockchain no es «una base de datos en la nube». Su valor está en tres propiedades que una base de datos normal no te da gratis: **integridad** (nadie altera el pasado sin que se note), **auditabilidad** (cualquiera verifica qué pasó y cuándo) y **no repudio** (quien firmó no puede negar que lo hizo). Si tu idea no aprovecha al menos una de las tres, probablemente no necesita blockchain — y el jurado lo va a notar.

El recorrido de una transacción

- 1 Tu **wallet** arma una instrucción («llama a esta función del contrato con estos datos») y la firma con tu clave privada.
- 2 La transacción se transmite a la red y un validador la incluye en un **bloque**.
- 3 La red ejecuta el código del contrato y actualiza el estado (por ejemplo, «el producto #4 quedó emitido por la marca»).
- 4 El bloque se **finaliza**: ya es parte permanente del historial. Tu wallet recibe un **recibo** (receipt) con el hash de la transacción.

5

Ese hash es la prueba pública: cualquiera lo abre en el **explorer** y ve qué pasó.

— 06 · FUNDAMENTOS

Glosario esencial

Los términos que van a volar el sábado. Cada uno con un ejemplo concreto, para que nadie del equipo se quede afuera de la conversación.

TÉRMINO	QUÉ ES	EJEMPLO CONCRETO
Wallet	App que guarda tus claves y firma transacciones. Es tu identidad y tu botón de «aceptar».	MetaMask, Rabby, Phantom.
Address	Identificador público de una wallet o de un contrato. Se puede compartir sin riesgo.	<code>0x4a3f...9c2b</code>
Private key	El secreto total. Quien la tiene, controla la cuenta. Jamás se comparte ni se sube a un repo.	64 caracteres hex. Nunca en el código.
Transaction (tx)	Instrucción firmada que cambia el estado de la cadena. Cuesta gas.	«Emitir el pasaporte del producto #5».
Gas	El costo computacional de ejecutar una tx. Se paga en el token nativo de la red.	En Monad se paga en <code>MON</code> .
Testnet	Red de pruebas. Idéntica a la real pero con tokens sin valor. Aquí construimos.	Monad Testnet, chain <code>10143</code> .
Faucet	Sitio que regala tokens de testnet gratis para poder pagar gas en pruebas.	<code>faucet.monad.xyz</code>
Smart contract	Programa desplegado en la cadena. Vive en una address y ejecuta su lógica cuando lo llaman.	Nuestro <code>OriginPass.sol</code> .
ABI	El «menú» del contrato: describe sus funciones para que el frontend sepa cómo llamarlas.	Un JSON que genera el compilador.
Event	Registro (log) que el contrato emite. Ideal para timelines y auditoría; barato de almacenar.	<code>ProductIssued(id)</code>

TÉRMINO	QUÉ ES	EJEMPLO CONCRETO
Explorer	Sitio web para ver bloques, transacciones y contratos. La prueba visual de la demo.	<code>testnet.monadexplorer.com</code>
Deploy	Publicar un contrato en la cadena por primera vez. Devuelve su address definitiva.	Se hace desde Remix o Hardhat.
Receipt	El comprobante que devuelve la red cuando una tx se confirma. Trae el hash y el estado.	El frontend lo espera para refrescar la UI.

07 · MONAD A FONDO

Qué es Monad y por qué es distinta

Monad es una blockchain **Layer-1 compatible con la EVM**: corre el mismo bytecode que Ethereum, así que Solidity, MetaMask, Hardhat, Foundry, Remix y Wagmi/Viem funcionan sin reescribir nada. La diferencia está en el motor: está rediseñada para ser **mucho más rápida**.

10.000 TPS

THROUGHPUT OBJETIVO

Transacciones por segundo, frente a las ~15–30 de Ethereum. Habilita apps con muchísimas interacciones.

~400 ms

TIEMPO DE BLOQUE

Un bloque nuevo cada ~0,4 s. La UI puede sentirse casi instantánea.

~800 ms

FINALIDAD

Tu transacción es definitiva en menos de un segundo, no en minutos.

La clave es que **todo ese rendimiento viene del software, no de exigir hardware caro**. Monad reescribió cuatro o cinco piezas del motor de Ethereum manteniendo intacta la superficie que toca el desarrollador. Para el hackathon no necesitas dominar esto, pero entenderlo te da munición para el pitch y para responder preguntas del jurado.

Las cinco piezas que la hacen rápida

1 MonadBFT

El mecanismo de **consenso** (cómo los nodos se ponen de acuerdo sobre el orden de las transacciones). Es una variante **pipelined** derivada de HotStuff/Jolteon que resuelve el problema de «tail-forking» y permite finalidad en una sola ronda.

2 Ejecución asíncrona

Separa el **consenso** de la **ejecución**. Mientras los nodos ya están votando el bloque N+1, otros ejecutan el bloque N. En Ethereum ambas cosas ocurren en la misma ronda y se estorban.

3 Ejecución paralela óptima

Ejecuta varias transacciones **a la vez** en distintos núcleos, asumiendo que no chocan; si dos tocan el mismo estado, detecta el conflicto y las re-ejecuta. El resultado final es idéntico al de ejecutarlas en serie.

4

MonadDB

Una base de datos hecha a medida para guardar el estado de Ethereum de forma eficiente en SSD. Es lo que evita que el disco se vuelva el cuello de botella a 10.000 TPS.

A esto se suma **RaptorCast**, el sistema de propagación de bloques de alta velocidad entre nodos. En conjunto: ordenar primero, ejecutar después, en paralelo, y guardar rápido.



LO QUE SÍ CAMBIA PARA TI COMO BUILDER

La ejecución paralela **no cambia cómo escribes la mayoría de los contratos**: el resultado siempre es equivalente a una ejecución serial, así que tu Solidity normal funciona igual.

Dos detalles prácticos sí importan:

- **Gas:** Monad cobra según el *gas limit* que pones, no según el gas realmente usado. No inflas los límites por reflejo — ponlos ajustados.
- **No hay mempool global:** evita diseños que dependan de leer transacciones «pendientes» de toda la red. Para una dApp normal esto es irrelevante.

— 08 · MONAD A FONDO · ACTUALIZACIÓN

Mainnet vs. Testnet: aclaración importante

La guía original asumía que Monad solo existía como testnet. **Eso quedó desactualizado:** Monad ya lanzó su **mainnet** (la red real, con valor económico) a finales de 2025. Ahora coexisten dos redes y conviene no confundirlas.

	MONAD TESTNET	MONAD MAINNET
Para qué sirve	Pruebas, hackathons, aprender. Es aquí donde construimos.	Aplicaciones reales con dinero real.
Chain ID	10143	143
El token MON	Sin valor real, gratis desde el faucet.	Tiene valor de mercado real.
Riesgo de error	Cero — si algo sale mal, no se pierde nada.	Alto — un bug puede costar dinero de verdad.

 REGLA DE ORO DEL SÁBADO

Todo el proyecto va sobre **Monad Testnet (chain ID 10143)**. Confirмен siempre que la wallet está en la red correcta antes de cada demo: el error más común y más bochornoso es desplegar o firmar en la red equivocada. Y aunque sea testnet, **usen wallets de prueba, nunca la principal**.

— 09 · MONAD A FONDO

Datos técnicos de la red

Los parámetros exactos para agregar Monad Testnet a la wallet. Cópienlos sin errar un solo carácter — un RPC o chain ID mal escrito es media hora perdida.

Network name	Monad Testnet
RPC URL	https://testnet-rpc.monad.xyz/
Chain ID	10143
Símbolo	MON

Explorer	https://testnet.monadexplorer.com/
Faucet	https://faucet.monad.xyz
Add network	https://faucet.monad.xyz/add-network

✓ATAJO

En lugar de teclear todo a mano, abran `faucet.monad.xyz/add-network` , conecten la wallet y dejen que agregue la red automáticamente. Verifiquen igual que el Chain ID quedó en `10143` .

10 · MONAD A FONDO

Monad vs. Ethereum vs. Solana

La frase que resume el posicionamiento de Monad: «**la velocidad de Solana, con las herramientas de Ethereum**». Esta tabla es oro para el pitch.

	ETHEREUM	SOLANA	MONAD
Modelo	L1, EVM	L1, no-EVM	L1, EVM
Throughput	~15–30 TPS	~1.000–3.000 TPS	~10.000 TPS
Tiempo de bloque	~12 s	~0,4 s	~0,4 s
Finalidad	~13 min	~2,5 s	~0,8 s
Lenguaje	Solidity	Rust / C	Solidity
Herramientas EVM	Nativas	No compatibles	Funcionan sin cambios
Ejecución paralela	No	Sí (Sealevel)	Sí (optimista)

I EL ARGUMENTO DE VENTA EN UNA LÍNEA

Solana es rápida pero te obliga a aprender Rust y a abandonar el ecosistema de Ethereum. Ethereum tiene todas las herramientas pero es lento y caro bajo carga. **Monad te da las dos cosas:** la familiaridad de la EVM con un rendimiento de otra liga. Por eso conviene a apps con muchas interacciones —juegos, pagos, agentes, coordinación— que en Ethereum serían inviables por costo o latencia.

— 11 · EJECUCIÓN

Roles del equipo de 3

La regla central: **solo una persona necesita saber Solidity**. El proyecto se divide de modo que los otros dos aporten en paralelo sin bloquearse esperando al contrato.



Emmanuel

CONTRATO · ON-CHAIN

Arquitectura on-chain, deploy y debugging.

Debe dominar

- ◆ Wallet, testnet, faucet, explorer, chain ID.
- ◆ Solidity mínimo: `struct`, `mapping`, `event`, `require`, visibilidad.
- ◆ Patrón: estado + eventos + getters + funciones write.
- ◆ Deploy con Remix (y Hardhat si sobra tiempo).
- ◆ Leer errores: red equivocada, falta MON, ABI vieja, nonce, revert.



Miguel Ángel Riveros

FRONTEND · WALLET · DEMO

La interfaz, la conexión de wallet y que la demo se vea.

Debe dominar

- ◆ React con Vite o Next.
- ◆ Conexión de wallet con RainbowKit / Wagmi.
- ◆ Leer contrato: `useReadContract`.
- ◆ Escribir: preparar llamada, enviar tx, esperar receipt, refrescar UI.
- ◆ Estados de UI: desconectado, red errónea, pendiente, confirmado, fallido.



Thomas Osorio

PRODUCTO · PITCH · QA

La historia, los datos de demo y que nada se rompa en vivo.

Debe dominar

- ◆ Traducir el problema a un flujo de 3 pantallas.
- ◆ Preparar datos de demo y el guion del pitch.
- ◆ Verificar cada transacción en el explorer.
- ◆ Tomar capturas/video por si falla internet.
- ◆ Completar el formulario de submission.



EL TRUCO DEL DESACOPLE

Mientras Emmanuel termina el contrato, Miguel Ángel puede avanzar la UI con datos «mock» y Thomas escribe el pitch. Cuando el contrato esté listo, solo se conectan la ABI y la address. Nadie espera de brazos cruzados.

Stack recomendado y qué NO hacer

La ruta más segura para un equipo que aprende sobre la marcha. Elegimos herramientas con buena documentación y máxima compatibilidad con la EVM.

CAPA	ELECCIÓN	POR QUÉ
Contrato	Solidity en Remix	Cero instalación, compila y despliega en el navegador. Hardhat solo si sobra tiempo.
Frontend	Vite + React o Next.js	Rápido de levantar, ecosistema enorme.
Web3	Wagmi + Viem + RainbowKit	Conexión de wallet y llamadas al contrato con hooks listos.
Estilos	Tailwind o CSS simple	No perder tiempo en CSS desde cero.
Deploy	Contrato en Testnet ; frontend local o Vercel	El frontend en Vercel solo si el flujo base ya funciona.
Repo	GitHub con README corto	Address del contrato, link del explorer, cómo correr la demo.

Qué NO intentar este sábado

Cada una de estas ideas es un agujero de tiempo que te impide terminar. El jurado premia un flujo completo, no una ambición incompleta.

~~Bridge propio~~

~~DEX completo~~

~~Lending protocol~~

~~NFT marketplace genérico~~

~~Datos personales on-chain~~

~~Contratos upgradeables / proxies~~

 LÍNEA ROJA DE SEGURIDAD

Nunca guardar **documentos, cédulas, correos ni datos personales** on-chain. Lo que va a la cadena es público y permanente. Si necesitas representar un documento, guarda solo su **hash** (una huella digital irreversible), nunca el contenido.

— 13 · EJECUCIÓN

Solidity mínimo

Solidity es el lenguaje de los contratos en la EVM. No hace falta ser experto: con este puñado de conceptos se construye el 90% de un MVP de hackathon. Pensá en un contrato como un objeto con **estado**, **funciones** que lo modifican y **eventos** que anuncian cada cambio.

PIEZA	PARA QUÉ SIRVE
<code>mapping(address => uint256)</code>	Una tabla clave-valor. La estructura de datos más usada: «para esta dirección, este número».
<code>struct</code>	Agrupar varios campos en una entidad (un «registro» con varios atributos).
<code>event</code>	Emite un log público. Es lo que el frontend y el explorer leen para armar timelines.
<code>require(cond, "msg")</code>	Valida una condición; si falla, revierte toda la transacción con ese mensaje. Tu cinturón de seguridad.
<code>msg.sender</code>	La address de quien llamó a la función. Base de todo control de permisos.
<code>public</code> / <code>view</code>	<code>public</code> : accesible desde fuera. <code>view</code> : solo lee, no cambia estado, no cuesta gas si se llama directo.
<code>external</code>	Función pensada para llamarse desde fuera del contrato (un poco más barata que <code>public</code>).
<code>calldata</code> / <code>memory</code> / <code>storage</code>	Dónde viven los datos: <code>calldata</code> = entrada barata e inmutable; <code>memory</code> = copia temporal; <code>storage</code> = el estado persistente del contrato.

I EL PATRÓN QUE SE REPITE

Casi todo contrato simple sigue la misma forma: **(1)** declarar el estado (`struct` + `mapping`), **(2)** funciones *write* que cambian ese estado y emiten un `event` , **(3)** `require` al inicio de cada write para validar permisos, **(4)** getters `view` para que el frontend lea. Si dominás ese molde, dominás el MVP.

— 14 · EJECUCIÓN

El contrato: `OriginPass.sol`

Un contrato real, completo y desplegable. Convierte el **serial que el producto ya tiene** en un pasaporte verificable: guarda solo el `hash` del serial (nunca el serial plano), controla qué marcas pueden emitir y deja un rastro público de emisión, revocación y reventa.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract OriginPass {
    enum Status { Active, Revoked }

    // Una entidad = el pasaporte de UNA unidad física
    struct Product {
        address issuer;           // la marca / emisor autorizado
        bytes32 serialHash;       // hash del serial existente, NO el serial
        bytes32 metadataHash;     // certificado, fotos, lote
        string productLine;       // "Sombrero vueltaio", "Sneaker X"...
        address owner;           // dueño actual (para reventa)
        Status status;
        uint256 issuedAt;
    }

    // Una marca autorizada (su nombre viaja para mostrar "emitido por...")
    struct Brand {
        bool allowed;
        string name;              // "Handoo Demo Brand", "Tejeduría Zenú"
        bytes32 metadataHash;     // logo, datos legales
    }

    address public admin;         // quién autoriza marcas
    uint256 public lastId;        // ids desde 1; 0 = no existe
    mapping(address => Brand) public brands; // emisores autorizados
    mapping(uint256 => Product) public products;
    mapping(bytes32 => uint256) public idBySerial; // serialHash -> id

    event BrandRegistered(address indexed brand, bool allowed);
    event ProductIssued(uint256 indexed id, address indexed issuer, bytes32 serialHash, bytes32
metadataHash);
    event ProductRevoked(uint256 indexed id, bytes32 reasonHash);
    event OwnershipTransferred(uint256 indexed id, address from, address to);

    constructor() { admin = msg.sender; }

    modifier onlyAdmin() { require(msg.sender == admin, "solo admin"); _; }
    modifier onlyIssuer() { require(brands[msg.sender].allowed, "no autorizado"); _; }
    modifier exists(uint256 id) { require(id != 0 && id <= lastId, "no existe"); _; }

    // El admin autoriza marcas y guarda su nombre (para "emitido por...")
    function registerBrand(address brand, string calldata name, bytes32 metadataHash, bool
allowed)
    external onlyAdmin
    {
        require(brand != address(0), "marca invalida");
        brands[brand] = Brand(allowed, name, metadataHash);
        emit BrandRegistered(brand, allowed);
    }
}
```

```

}

// Una marca autorizada emite el pasaporte de UNA unidad
function issueProduct(bytes32 serialHash, bytes32 metadataHash, string calldata productLine)
    external onlyIssuer returns (uint256 id)
{
    require(serialHash != bytes32(0), "serial invalido");
    require(metadataHash != bytes32(0), "metadata invalida");
    require(idBySerial[serialHash] == 0, "serial ya emitido");
    id = ++lastId;
    products[id] = Product(
        msg.sender, serialHash, metadataHash, productLine, msg.sender, Status.Active,
        block.timestamp
    );
    idBySerial[serialHash] = id;
    emit ProductIssued(id, msg.sender, serialHash, metadataHash);
}

// Solo el emisor original revoca (lote defectuoso, robo, error)
function revokeProduct(uint256 id, bytes32 reasonHash) external exists(id) {
    Product storage p = products[id];
    require(msg.sender == p.issuer, "solo el emisor");
    p.status = Status.Revoked;
    emit ProductRevoked(id, reasonHash);
}

// Reventa (OPCIONAL, post-MVP): el dueño actual transfiere
function transferOwnership(uint256 id, address newOwner) external exists(id) {
    Product storage p = products[id];
    require(msg.sender == p.owner, "solo el dueño");
    require(newOwner != address(0), "destino invalido");
    emit OwnershipTransferred(id, p.owner, newOwner);
    p.owner = newOwner;
}

// Verificación pública por escaneo: del serial al pasaporte
function verifyBySerial(bytes32 serialHash) external view returns (Product memory) {
    uint256 id = idBySerial[serialHash];
    require(id != 0, "no encontrado");
    return products[id];
}

function getProduct(uint256 id) external view exists(id) returns (Product memory) {
    return products[id];
}
}

```

! PASO 0 ANTES DE EMITIR

Tras el deploy hay que **autorizar la marca** o `issueProduct` revierte con «no autorizado». Es lo primero de la demo:

```
registerBrand(adminWallet, "Handoo Demo Brand", brandMetadataHash, true) .
```

✓ POR QUÉ ESTE CONTRATO ES BUEN MATERIAL DE HACKATHON

- **Autorización con nombre de marca:** el `admin` habilita emisores y guarda su nombre, así la verificación pública dice «**emitido por Handoo Demo Brand**», no una wallet suelta.
- **Verificación por escaneo:** `idBySerial` conecta el QR/serial con el pasaporte en una sola lectura `view` (gratis).
- **Privacy-aware:** viaja el `hash` del serial, nunca el serial plano ni datos del comprador.
- **Con guardas reales:** marca válida, serial y metadata no nulos, serial no duplicado, el id debe existir y solo el emisor revoca.
- **Core de la demo = 3 acciones:** emitir → verificar → revocar. `transferOwnership` queda como opcional si sobra tiempo.

— 15 · EJECUCIÓN

Frontend: conectar la UI con el contrato

Con **Wagmi** (hooks de React) + **Viem** (cliente de bajo nivel) + **RainbowKit** (botón de conexión), hablar con el contrato son tres pasos: **configurar la red, leer y escribir**.

0 • Montar el proyecto (terminal)

● ● ● terminal

BASH

```
# 1. crear el proyecto (React + Vite)
npm create vite@latest originpass -- --template react-ts
cd originpass

# 2. dependencias web3
npm install wagmi viem @rainbow-me/rainbowkit @tanstack/react-query

# 3. levantar el dev server → http://localhost:5173
npm run dev
```

I DÓNDE VA LA ABI Y LA ADDRESS

Tras compilar en Remix, copia la **ABI** (pestaña «Compilation Details» → ABI) a un archivo `src/abi/originPass.ts` y la **address** que devuelve el deploy a una constante `ORIGINPASS_ADDRESS`. Cada vez que redespliegues el contrato, **vuelve a copiar ambas** — una ABI vieja es el bug #1 de las demos.

1 • Definir la red Monad

● ● ● chains.ts

TYPESCRIPT

```
import { defineChain } from 'viem';

export const monadTestnet = defineChain({
  id: 10143,
  name: 'Monad Testnet',
  nativeCurrency: { name: 'Monad', symbol: 'MON', decimals: 18 },
  rpcUrls: { default: { http: ['https://testnet-rpc.monad.xyz/'] } },
  blockExplorers: {
    default: { name: 'Monad Explorer', url: 'https://testnet.monadexplorer.com' },
  },
  testnet: true,
});
```

2 · Leer datos del contrato

● ● ● ProductCard.tsx

REACT

```
const { data: product } = useReadContract({
  address: ORIGINPASS_ADDRESS,
  abi: originPassAbi,
  functionName: 'getProduct',
  args: [id],
});

// product.status → 0 Active · 1 Revoked
// escaneo del comprador: functionName 'verifyBySerial', args [serialHash]
```

3 · Escribir (enviar una transacción)

● ● ● IssueButton.tsx

REACT

```
const { writeContract, data: hash, isPending } = useWriteContract();

// esperar el receipt y refrescar la UI cuando confirme
const { isLoading: confirming, isSuccess } =
  useWaitForTransactionReceipt({ hash });

function emitirPasaporte(serialHash, metadataHash, productLine) {
  writeContract({
    address: ORIGINPASS_ADDRESS,
    abi: originPassAbi,
    functionName: 'issueProduct',
    args: [serialHash, metadataHash, productLine],
  });
}

// isPending → esperando firma en la wallet
// confirming → tx enviada, esperando confirmación on-chain
// isSuccess → confirmada: ya se puede mostrar el link al explorer
```

Los cinco estados de UI que no pueden faltar

El error más común en demos es no manejar estos estados: el usuario hace clic y «no pasa nada». Cada estado necesita su propio feedback visual.

ESTADO	QUÉ MOSTRAR AL USUARIO
Disconnected	Botón «Conectar wallet». Nada más es accionable.
Wrong network	Aviso «Cambia a Monad Testnet» + botón que dispara el cambio de red.
Pending	Spinner: «Confirma en tu wallet...». La firma está en manos del usuario.
Confirmed	Check verde + link directo a la tx en el explorer. Este es el momento estrella de la demo.
Failed	Mensaje de error legible (no el stack crudo) + botón «Reintentar».

! NO HAGAS POLLING AGRESIVO

Para muchas lecturas, usa **multicall** o un indexer en lugar de consultar cada segundo. En el hackathon basta con refrescar tras cada transacción confirmada — pedir datos en bucle cada segundo satura el RPC y ralentiza la demo.

— 16 · EL PROYECTO

Idea recomendada: Handoo OriginPass

Un **pasaporte de autenticidad por unidad** para productos físicos. Cualquier empresa convierte el **serial que ya usa** en un certificado público y verificable en Monad — sin rediseñar su fabricación. Arrancamos con un caso estrecho (un sombrero vueltiao / Tejeduría Zenú) y el mismo modelo escala a moda, lujo, sneakers y exportación.

El problema

La falsificación es un mercado de **~USD 467.000 millones** (OCDE/EUIPO, datos 2021). Compradores, retailers y aduanas no distinguen fácil un original de una copia AAA; ropa, calzado y cuero son el **62%** de lo incautado.

La solución

La marca toma el **serial existente** de cada unidad, se calcula su `hash` y se registra como pasaporte en Monad. La app genera un **QR**; el comprador escanea y verifica emisor, estado e historial.

Por qué Monad

Certificar **por unidad** a escala global son millones de registros: pide alto throughput, baja latencia y bajo costo. **Este caso no tiene sentido en una blockchain lenta o cara.** Monad lo resuelve en una **L1 EVM pública** (10.000 TPS, 400 ms, 800 ms) sin encerrar a la marca en una cadena privada.

Cómo funciona la verificación

- 1 El **admin** autoriza a la marca como emisor (`registerBrand`).
- 2 La marca **emite el pasaporte** de cada unidad: se guarda el `hash` del serial + metadatos (`issueProduct`).
- 3 La app imprime o asocia un **QR** (o NFC) al producto.
- 4 El comprador **escanea**: la app calcula el hash y consulta Monad (`verifyBySerial`).
- 5 Ve en segundos **la marca emisora (p. ej. «Handoo Demo Brand»), estado (activo/revocado), línea de producto y la tx en el explorer** — sin confiar en el vendedor.

I SOBRE EL HASH DEL SERIAL

MVP: hash de un serial de prueba. **Producción:** combinar serial + marca + un **salt** del backend (o NFC con challenge) para evitar enumeración o clonación simple del QR.

I EL PITCH DE 60-90 SEGUNDOS

«Las falsificaciones mueven cientos de miles de millones al año — la OCDE/EUIPO estima USD 467.000 millones. Handoo OriginPass permite que cualquier empresa certifique cada producto usando el serial que ya tiene: lo convertimos en un hash, lo registramos en Monad y generamos una verificación pública por QR. El comprador no confía en el vendedor: verifica en segundos que la unidad fue emitida por la marca, que no fue revocada y que su certificado existe en una blockchain pública. Como es certificación por millones de unidades, necesita una red rápida y barata: por eso Monad.»

Por qué puede ganar — y su riesgo

A favor

- ◆ Problema enorme y entendible en 20 s, con cifra de apertura potente.
- ◆ Argumento de Monad creíble: volumen masivo por unidad.
- ◆ No exige cambiar la fabricación: usa el serial existente.
- ◆ Demo muy visual: emitir → escanear → original vs. falso.
- ◆ Caso de arranque local y concreto (Tejeduría Zenú).

En contra / a cuidar

No prometer que blockchain elimina la falsificación física: **verifica el certificado, no el átomo**. Hay competidores en lujo (p. ej. Aura). Se compensa con honestidad de alcance (ver §17) y una demo que muestre el «original vs. fake» en vivo.

— 17 · EL PROYECTO

Alcance honesto, riesgos y estrategia de entrada

Lo que hace defendible este pitch ante un jurado técnico no es prometer magia, sino marcar con precisión **qué prueba y qué no prueba** la blockchain — y a quién le venderíamos primero.

! EL LÍMITE QUE HAY QUE DECIR EN VOZ ALTA

Blockchain **no verifica la realidad física por sí sola**. OriginPass prueba que un **emisor autorizado registró** ese serial, que **no fue revocado** y que su historial es público. La unión con el objeto se hace con QR/NFC/sello. Si un falsificador clona serial + QR de una unidad real, puede crear un gemelo visual: por eso en producción se sube la seguridad con **NFC, sello anti-manipulación, challenge criptográfico o detección de escaneos sospechosos**. Decir esto *suma* credibilidad, no resta.

Decisiones de alcance para el sábado

Emitir pasaporte

Revocar

Verificar por QR/serial

Transferir propiedad (opcional)

~~Marketplace~~~~Aduanas~~~~Supply chain completo~~~~NFC real / chip~~

El wedge: a quién venderle primero

No se le vende a Rolex en un hackathon (úsalo solo como *ejemplo narrativo*: «una marca premium de relojes»). El mercado de entrada real son empresas con dolor de falsificación pero **sin el lock-in** de los gigantes del lujo:

1

Artesanos exportadores

Denominación de origen (Tejeduría Zenú, café, artesanía). Dolor real de copias y narrativa cultural fuerte.

2 Streetwear y sneakers

Reventa intensa donde la autenticidad **eleva el precio**. Comprador joven, ya crypto-friendly.

3 Marcas premium medianas

Relojes independientes, moda de autor. Quieren proteger marca sin montar su propia cadena.

✓ DE DÓNDE SALE EL CONTRATO

OriginPass nace de tomar el `TodoList.sol` del workshop y reescribir conceptos hacia una historia real: `Task` → `Product` (la unidad), `createTask` → `issueProduct` (emisión), y se le añade **control de emisores** (`registerBrand`) y **verificación por serial** (`verifyBySerial`), con un evento por paso para el explorer.

— 18 · EL SÁBADO · ANTES

Labs antes del evento

No se aprende a manejar wallets el día del hackathon. Estos cuatro labs, hechos esta semana, son la diferencia entre llegar a construir y llegar a pelear con la configuración.

0

Wallet y faucet

~20 MIN · LOS 3

1. Crear una wallet de prueba (no la principal).
2. Agregar Monad Testnet.
3. Pedir **MON** en el faucet.
4. Enviar una tx pequeña a otro integrante.
5. Abrirla en el explorer.



LISTO CUANDO

los tres tienen **MON** y saben ver un tx hash.

1

Remix

~45 MIN · EMMANUEL

1. Abrir **TodoList.sol** del workshop.
2. Compilar.
3. Deploy con Injected Provider en Monad Testnet.
4. Ejecutar funciones write.
5. Copiar address + ABI.



LISTO CUANDO

hay un contrato vivo y una tx visible.

2

Frontend

~90 MIN · MIGUEL ÁNGEL

1. Clonar el workshop de DevLabX3.
2. `npm install`.
3. Configurar address + ABI.
4. Conectar wallet.
5. Leer la lista; crear un ítem desde la UI.



LISTO CUANDO

un compañero sin blockchain puede usar la dApp.

3

Idea propia

~2 H · LOS 3

Tomar `TodoList` y convertirlo a un caso real: renombrar la entidad, la acción principal y el cierre/verificación, y emitir un evento por paso (ver §17).



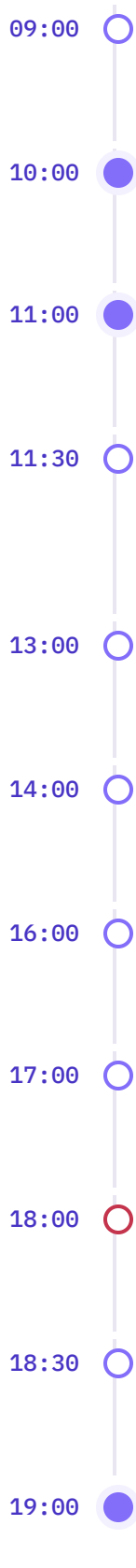
LISTO CUANDO

el contrato refleja una historia de usuario, no un CRUD genérico.

— 19 · EL SÁBADO

Plan hora a hora

Un plan de batalla concreto para las ~7 horas de hacking. La consigna que lo gobierna todo: **no agregar features nuevas hasta que el flujo base funcione de punta a punta.**

- 
- A vertical timeline on the left side of the page, consisting of a light purple line with circular markers. The markers are colored purple for most items and red for the 'Code freeze' item. The timeline starts at 09:00 and ends at 19:00.
- 09:00**  **Llegar y montar base**
Preregistro, WiFi, enchufes. Confirmar que los 3 tienen wallet y **MON**.
 - 10:00**  **Reglas y Monad 101**
Escuchar las restricciones exactas de GitHub y submission. Anotar.
 - 11:00**  **Decidir la idea final**
Si no hay nada mejor, OriginPass. Escribir user story, contrato mínimo y pantallas.
 - 11:30**  **Arranque en paralelo**
Emmanuel: contrato + deploy inicial. Miguel Ángel: skeleton del frontend. Thomas: copy y datos de demo.
 - 13:00**  **Almuerzo con foco**
Revisar riesgos y recortar alcance. No abrir el laptop a media comida.
 - 14:00**  **Integración UI ↔ contrato**
Conectar ABI + address. Que el flujo base funcione antes que cualquier extra.
 - 16:00**  **Pulir**
UX, manejo de errores, links al explorer, README.
 - 17:00**  **Ensayo de pitch + backup**
Practicar la demo. Grabar un video corto por si falla internet.
 - 18:00**  **Code freeze**
Cero cambios en GitHub.
 - 18:30**  **Entregar en la app**
Nombre, descripción, repo, contrato, demo, integrantes.
 - 19:00**  **Demos**
Abrir la demo ya cargada ANTES de pasar al frente.

— 20 · EL SÁBADO

Checklists operativos

Antes de ir

- ☒ Laptop cargado + cargador.
- ☒ Node 18+ o 20+.
- ☒ Git y VS Code.
- ☒ Wallet de prueba creada.
- ☒ Monad Testnet agregada.
- ☒ **MON** del faucet en las 3 wallets.
- ☒ Cuenta de GitHub lista.
- ☒ Template frontend corre local.
- ☒ Remix probado.
- ☒ Hotspot del celular de respaldo.

Submission

- ☒ Nombre del proyecto.
- ☒ Descripción en 1 frase.
- ☒ Repo público.
- ☒ URL de demo o instrucciones local.
- ☒ Contract address.
- ☒ Link del explorer.
- ☒ Capturas o video de backup.
- ☒ Integrantes.

Seguridad

- ☒ Nunca subir `.env`.
- ☒ Nunca usar la wallet principal.
- ☒ Ninguna private key en el frontend.
- ☒ Cero datos personales on-chain.
- ☒ Si usan hashes de seriales o documentos, usen datos de prueba.

— 21 · EL SÁBADO

Riesgos y errores comunes en vivo

Los tropiezos que se llevan media hora de un equipo en pleno hackathon — todos prevenibles si los conocen de antemano.

SÍNTOMA	CAUSA PROBABLE Y ARREGLO
La tx falla o «no pasa nada»	Wallet en la red equivocada . Verificar Chain ID <code>10143</code> antes de cada acción.
«Insufficient funds» al firmar	Falta <code>MON</code> de gas. Volver al faucet. Tener saldo en las 3 wallets.
El frontend lee datos viejos o falla	ABI desactualizada tras redespargar. Re-copiar ABI + address cada vez que cambie el contrato.
«Nonce too high / too low»	La wallet perdió el conteo. En MetaMask: reset de cuenta o esperar a que sincronice.
El contrato revierte sin mensaje claro	Falló un <code>require</code> . Leer el motivo en el explorer; suele ser un permiso (<code>msg.sender</code> incorrecto).
Address mal copiada	Un carácter de más/menos. Copiar siempre desde el explorer o desde Remix, nunca a mano.
Se cae el internet en la demo	Por eso el video de backup y el hotspot. Abrir la demo cargada antes de subir al frente.
Gas limit exagerado	Monad cobra por gas limit, no por gas usado. No inflar los límites por reflejo.

! LA REGLA DE LOS NERVIOS

El 80% de los fallos en vivo son **red equivocada** o **falta de gas**. Antes de cada demo, dos chequeos de 5 segundos: ¿estoy en Monad Testnet? ¿tengo **MON** ? Hecho eso, respiren.

★ IMPRIMIR APARTE

Hoja de acción del sábado

Una sola página. Lo mínimo para operar sin abrir el manual completo. Imprímanla y péguenla en la mesa.

RED MONAD TESTNET

RPC	testnet-rpc.monad.xyz
Chain ID	10143
Símbolo	MON
Explorer	testnet.monadexplorer.com
Faucet	faucet.monad.xyz

ROLES

- **Emmanuel** — contrato, deploy, debugging.
- **Miguel Ángel** — frontend, wallet, demo.
- **Thomas** — pitch, datos, QA, submission.

COMANDOS CLAVE

```
npm create vite@latest app -- --template react-ts
npm i wagmi viem @rainbow-me/rainbowkit
npm i @tanstack/react-query
npm run dev → localhost:5173
```

REMIX (DEPLOY)

1. Compilar el .sol.
2. Environment → **Injected Provider**.
3. Wallet en Monad Testnet (10143).
4. Deploy → copiar **address + ABI**.

PASO 0 DE LA DEMO

Tras el deploy: `registerBrand(admin, "Handoo Demo Brand", hash, true)` — sin esto, `issueProduct` revierte por «no autorizado».

PITCH EN UNA LÍNEA

«Handoo OriginPass convierte el serial que cada producto ya tiene en un pasaporte verificable en Monad: el comprador escanea un QR y confirma que la unidad fue emitida por la marca y no fue revocada.»

SI ALGO FALLA EN VIVO

¿Red correcta? Chain 10143.
 ¿Hay gas? MON del faucet.
 ¿ABI vieja? recopiar tras redeploy.
 ¿Revert? leer motivo en explorer.

ENTREGA (18:30)

- Nombre + 1 frase.
- Repo público.
- Contract address + link explorer.
- Demo URL / video backup.
- Integrantes.

NO OLVIDAR

11:00 nada en GitHub antes · **18:00** code freeze · wallet de prueba (no la principal) · cero datos personales on-chain · abrir la demo cargada antes de pasar al frente.

— 22 · REFERENCIAS

Fuentes

Datos del evento verificados al 04 jun 2026; datos técnicos de Monad enriquecidos con documentación oficial y análisis recientes (2025–2026).

Monad Colombia · MonadBlitz Medellín

monadcolombia.xyz

Monad Events

monad.xyz/events

Luma · MonadBlitz Medellín

luma.com/o56ekpyb

DevLabX3 · repo del workshop

github.com/devlabx3/Med_BlockChain_WorkShop1

Monad Docs · introducción y arquitectura

docs.monad.xyz

Monad Docs · network information (mainnet & testnet)

docs.monad.xyz/developer-essentials/network-information

Monad Docs · best practices (gas, mempool)

docs.monad.xyz/developer-essentials/best-practices

Monad Docs · gas pricing

docs.monad.xyz/developer-essentials/gas-pricing

Monad Docs · testnets

docs.monad.xyz/developer-essentials/testnets

OCDE/EUIPO · comercio global de falsificaciones (USD 467B, datos 2021)

[oecd.org · Mapping Global Trade in Fakes](https://oecd.org/Mapping-Global-Trade-in-Fakes)

Monad · Parallel Execution (anuncio oficial)

monad.xyz/announcements/parallel-execution-monad

Faucet · add network

faucet.monad.xyz/add-network

Guía de preparación para MonadBlitz Medellín · documento interno del equipo · **no oficial**. Confirмен reglas, premios y restricciones definitivas en la apertura del evento (10:00). Los parámetros de red y rendimiento provienen de la documentación de Monad y pueden cambiar; verifíquenlos el mismo día.